

Kubernetes Troubleshooting Guide

A symptom-driven guide for RDAS in the `rdas` namespace. Start with the triage commands, then jump to the matching section.

Triage

```
kubectl -n rdas get pods
kubectl -n rdas get events --sort-by=.lastTimestamp | tail -30
kubectl -n rdas logs deploy/rdas-app --tail=100
kubectl -n rdas describe pod <pod>
```

1. App pod not Ready

Readiness checks `db + readinessState` (SOAP is intentionally excluded), so a not-Ready app pod almost always means it cannot reach MySQL.

```
kubectl -n rdas get pods -l app=mysql
kubectl -n rdas logs deploy/rdas-app | grep -i -E "HikariPool|jdbc|Communications|Access denied"
```

- `Communications link failure / Unknown host mysql`: the MySQL Service is not up, or in external mode `external-mysql.yaml` was not applied / points at the wrong host. Check `kubectl -n rdas get svc mysql`.
- `Access denied for user`: the Secret credentials do not match the database. Re-check `k8s/secret.yaml`, then `rollout restart` both MySQL and the app.
- `Probe failing during the initial harvest`: this is normal only for overall health, not readiness. If readiness itself fails, the cause is the DB.

2. Pod CrashLoopBackOff

```
kubectl -n rdas logs <pod> --previous | tail -50
```

- `Unable to determine Dialect / Hikari init exception: database unreachable at startup`. Fix MySQL first; the app will start once the DB is healthy.

- `OOMKilled` (`kubectl -n rdas describe pod <pod>`): raise memory limits in `deployment.yaml`. The JVM honours the container limit via `MaxRAMPercentage`.

3. ImagePullBackOff / ErrImageNeverPull

The image is not present in the cluster.

```
kubectl -n rdas describe pod <pod> | grep -A3 Events
```

- minikube: build inside `eval $(minikube docker-env)`.
- kind: `kind load docker-image reference-data-aggregation-service:1.0.0`.
- Keep `imagePullPolicy: IfNotPresent` for local images; use a registry + pull secret otherwise.

4. Catalog is empty / health is DOWN

Overall health is `DOWN` while `countryCount` is 0; readiness stays `UP`.

```
kubectl -n rdas port-forward svc/rdas-app 8080:80
curl http://localhost:8080/api/v1/admin/catalog/status
curl http://localhost:8080/actuator/health | jq .components.catalog
kubectl -n rdas logs deploy/rdas-app | grep -i harvest
```

- Just started: wait 2-3 minutes for the first harvest to complete.
- `lastRefreshSuccessful=false` with a SOAP error: the cluster cannot reach the SOAP endpoint - check egress / network policy / DNS. Trigger a retry with `POST /api/v1/admin/catalog/refresh`.
- Circuit breaker open: see section 6.

5. Catalog is stale

`stale=true` means data is older than `RDAS_HARVEST_STALE_AFTER_HOURS`. Reads still work (this is the resilience design). Inspect why refresh is not succeeding:

```
curl http://localhost:8080/api/v1/admin/catalog/status # lastError,
lastRefreshCompletedAt
kubectl -n rdas logs deploy/rdas-app | grep -i -E "harvest|SOAP"
```

Common cause: the upstream SOAP service is down or rate-limiting. Confirm with the circuit-breaker metric and retry off-peak.

6. SOAP failures / circuit breaker open

```
curl http://localhost:8080/actuator/health | jq .components.circuitBreakers
curl -s http://localhost:8080/actuator/prometheus | grep resilience4j_circuitbreaker_state
```

- `state="open"` : the breaker tripped after sustained failures and is fast-failing. It auto-transitions to half-open after the wait window and closes on success. No action needed unless it stays open - then the upstream is genuinely down; reads continue to serve from the catalog.
- Hitting the provider rate limit (`limit-for-period 90/min`) shows up as rate limiter rejections; harvesting paces itself, so this is self-correcting.

7. HPA not scaling

```
kubectl -n rdas get hpa rdas-app
kubectl top pods -n rdas
```

- `targets: <unknown>/70%` : `metrics-server` is not installed or not ready. Install it; on Docker Desktop/kind it often needs `--kubelet-insecure-tls` .
- HPA needs CPU `requests` set (they are, in `deployment.yaml`) to compute utilisation.

8. Ingress returns 404 / not reachable

```
kubectl -n rdas get ingress rdas-ingress
kubectl get pods -n ingress-nginx
```

- No ingress controller installed: install ingress-nginx, or just use `kubectl port-forward svc/rdas-app 8080:80` .
- Host mismatch: the request `Host` must be `rdas.local` (the rule host). Add it to `/etc/hosts` mapped to the ingress address.

9. Useful one-liners

```
kubectl -n rdas exec deploy/rdas-app -- wget -qO-
    http://localhost:8080/actuator/health/readiness
kubectl -n rdas exec -it statefulset/mysql -- mysql -urdas -prdas rdas -e "select count(*)
    from country;"
kubectl -n rdas rollout restart deployment/rdas-app
```